AÑO 1 · TÉRMINO 1 · MÓDULO 3 · SEMANA 1

Introducción a la gestión de la ingeniería de software

El software lo construyen equipos, cambia constantemente y debe ser trazable. Esta semana aprendes cómo se organiza el desarrollo como proceso, cómo la gestión de configuración mantiene el producto bajo control y cómo Git y GitHub permiten trabajar sobre el mismo código sin perder cambios.

Guía de estudio | 8 lecciones | 600 min

Contenido

1. Qué es la ingeniería de software
2. El ciclo de vida de desarrollo y los modelos de proceso
3. Ingeniería de requisitos: de deseos a especificaciones
4. Modelar software: de los diagramas de contexto a UML
5. Gestión de la configuración del software
6. Control de versiones
7. Fundamentos de Git: áreas, commits y ramas
8. GitHub, remotos y flujos de colaboración
9. Soluciones resueltas
10. Glosario
11. Cuestionario de práctica

Objetivos de aprendizaje

- Explicar la diferencia entre programar e ingeniería de software.
- Describir un ciclo de vida de desarrollo y dos modelos de proceso.
- Escribir requisitos comprobables y distinguir los funcionales de los no funcionales.
- Leer y bosquejar modelos básicos: diagrama de contexto, modelo ER y máquina de estados.
- Definir la gestión de configuración del software y sus actividades principales.
- Usar comandos básicos de Git para inicializar, preparar, confirmar, crear ramas y fusionar.
- Conectar un repositorio local con un remoto y describir un flujo con solicitudes de cambio.

01 Lección 1 · 15 min

Qué es la ingeniería de software

Programar es escribir instrucciones que una computadora puede ejecutar. La ingeniería de software añade la disciplina necesaria para entregar un sistema que funcione, que pueda cambiarse y que las personas que dependen de él entiendan. La diferencia importa porque la mayoría de los fallos de software no son errores de tecleo: vienen de requisitos poco claros, mala planificación, documentación ausente y falta de comunicación entre el cliente y quienes construyen el sistema.

Una forma útil de ver la brecha es una cadena de traducciones. El cliente tiene una necesidad real. La describe de forma imperfecta. Ventas promete algo. Los analistas escriben requisitos. Los diseñadores proponen una estructura. Los programadores escriben código. Cada paso puede perder significado. La ingeniería de software intenta que cada paso sea explícito y comprobable, con especificaciones escritas, revisiones y pruebas.

El software además es un producto especial. Es intangible, cambia con frecuencia y casi nunca está terminado. Por eso los ingenieros trabajan con modelos: descripciones simplificadas de un sistema, como un diagrama de flujo de datos o una lista de casos de uso. Un buen modelo es lo bastante abstracto para ser simple, lo bastante preciso para confiar en él y lo bastante barato para cambiarlo antes de escribir código.

Piénsalo así

Construir una casa necesita planos antes que ladrillos. El software necesita planes y modelos antes que código, porque cambiar un muro en el papel cuesta mucho menos que derribarlo.

Ideas clave

- La ingeniería de software trata de entregar y mantener software, no solo de escribirlo.
- La mayoría de los problemas de proyecto vienen de brechas de comunicación y planificación.
- Los modelos permiten discutir y verificar un sistema antes de construirlo.

Errores comunes

- ✗ Creer que "compila" significa "cumple la necesidad".
  ✓ **Solución:** Compara el resultado con requisitos escritos y pruebas.

Ejercicios

1.1 Encuentra la brecha Nivel 1

Un cliente dice "haz el reporte más rápido". Escribe dos preguntas que harías antes de programar.

Pista: ¿Qué significa "más rápido" para ellos: segundos, minutos o otro conjunto de datos?

1.2 Escribe un requisito comprobable Nivel 2

Reescribe "el inicio de sesión debe ser seguro" como un requisito con una condición medible.

Pista: Nombra la condición, el actor y el resultado esperado.

Lección 2 · 20 min

02 El ciclo de vida de desarrollo y los modelos de proceso

Un proceso de desarrollo de software define qué se hace, quién lo hace, cuándo y cómo. Todo proceso realiza cuatro actividades generales: especificar el software, desarrollarlo, validarlo y evolucionarlo después de la entrega. El ciclo de vida es la secuencia de fases por las que pasa un producto: análisis de requisitos, diseño, implementación, pruebas, despliegue y mantenimiento.

Un proceso en cascada ejecuta estas fases una sola vez, en orden. Es fácil de planificar, pero lento para reaccionar al cambio, y los problemas descubiertos en las pruebas son caros de corregir. Un proceso iterativo e incremental divide el proyecto en iteraciones. Cada iteración planifica, analiza, diseña, construye y prueba una parte pequeña, y produce una línea base: una versión funcional y parcialmente completa del sistema. La diferencia entre dos líneas base es un incremento.

Los procesos de tipo unificado organizan las iteraciones en fases: inicio (objetivos y alcance), elaboración (arquitectura y riesgos principales), construcción (creación del sistema) y transición (entrega). Los métodos adaptativos, como Scrum, aplican la misma idea con ciclos cortos y retroalimentación frecuente de los usuarios. En todos los casos, la pregunta clave es cuán pronto el equipo aprende algo que sería caro descubrir tarde.

Piénsalo así

Piensa en una clase de cocina. Cascada es seguir una receta de principio a fin sin probar. El trabajo iterativo es probar la salsa después de cada paso y ajustar el siguiente.

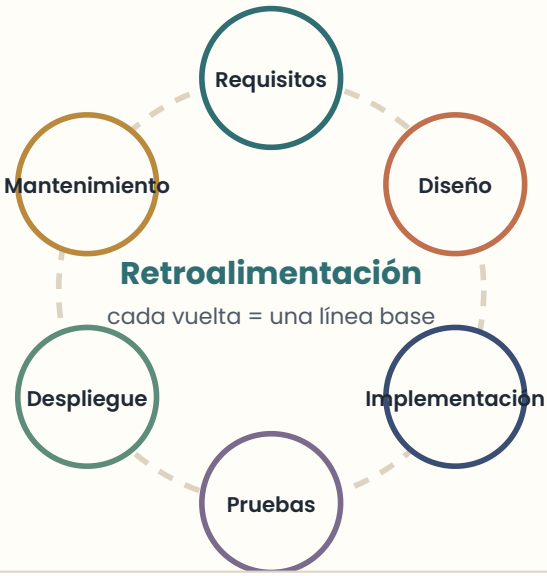


Diagrama · El ciclo de vida de desarrollo y los modelos de proceso

Ideas clave

- Un modelo de proceso es una descripción simplificada de cómo trabajan los equipos reales.
- El trabajo iterativo produce líneas base; la diferencia entre ellas es un incremento.
- Elige un proceso según cuánto cambio esperas y cuán costosos son los errores tardíos.

Errores comunes

- ✗ Llamar "ágil" a un proyecto en cascada solo porque tiene reuniones.
  ✓ **Solución:** Pregunta si la retroalimentación cambia el plan durante el proyecto.

Ejercicios

2.1 Elige un modelo Nivel 1

Un formulario oficial tiene reglas legales fijas que no cambiarán en dos años. ¿Qué modelo encaja mejor y por qué?

Pista: Piensa cuánto cambio esperas después de firmar los requisitos.

2.2 Dibuja tu propio historial de líneas base Nivel 2

Un equipo construye una app de chat en tres iteraciones: inicio de sesión, mensajes y notificaciones. Nombra la línea base y el incremento tras la iteración 2.

Pista: Una línea base es una versión funcional; el incremento es la diferencia con la anterior.

Lección 3 · 25 min

03 Ingeniería de requisitos: de deseos a especificaciones

Un requisito es una capacidad o condición que un sistema debe tener para ser aceptado por sus usuarios, su cliente o una norma. La ingeniería de requisitos convierte un deseo de negocio en enunciados precisos y comprobables que comparten analistas, diseñadores, programadores y testers. Los errores aquí son los más caros, porque todas las fases posteriores se construyen sobre ellos.

El trabajo pasa por siete actividades que a menudo se solapan. El inicio identifica la necesidad y una solución de alto nivel. La obtención (elicitación) recoge las necesidades de los interesados. La elaboración las refina en escenarios, reglas de negocio y flujos principales y alternativos. La negociación concilia visiones en conflicto y evalúa riesgo, costo y plazo. La especificación lo pone todo por escrito en una plantilla estándar, a menudo con diagramas y prototipos. La validación revisa la especificación con los usuarios y corrige problemas antes de construir. La gestión registra cada cambio de los requisitos durante todo el proyecto.

La obtención es difícil por tres motivos. Problemas de alcance: los límites del sistema no están claros, así que el analista escribe supuestos y restricciones que dicen qué entra y qué no. Problemas de comprensión: los usuarios pueden no saber lo que necesitan o describirlo a un nivel demasiado alto. Volatilidad: los requisitos cambian de una reunión a otra, así que los equipos registran los acuerdos en actas y trabajan con un usuario representativo.

Los requisitos tienen capas. Los requisitos de negocio expresan los objetivos de la organización. Los requisitos de los interesados describen las necesidades de un grupo de usuarios. Los requisitos de la solución dicen qué debe hacer el sistema y se dividen en funcionales (comportamiento: "el sistema envía un recibo por correo tras el pago") y no funcionales (cualidades: "una página carga en menos de 2 segundos con 500 usuarios conectados"). Los requisitos de transición son capacidades temporales para pasar de la situación anterior a la nueva, como migrar datos antiguos.

> **Piénsalo así**
> Un sastre mide dos veces y corta una. Los requisitos son las medidas: si están mal, ninguna costura cuidadosa (programación) hará que el traje quede bien.



Diagrama · Ingeniería de requisitos: de deseos a especificaciones

Ideas clave

- Un requisito debe ser comprobable: alguien debe poder decir "se cumple" o "no se cumple".
- Siete actividades: inicio, obtención, elaboración, negociación, especificación, validación y gestión.
- Funcional = qué hace el sistema. No funcional = qué tan bien lo hace.

Errores comunes

- ✗ Escribir "el sistema debe ser rápido y fácil".
  ✓ **Solución:** Añade una medida: "los resultados de búsqueda aparecen en menos de 1 segundo".
- ✗ Cambiar requisitos por chat sin dejar registro.
  ✓ **Solución:** Registra cada acuerdo y quién lo aprueba; eso es gestión de requisitos.

Ejercicios

3.1 Encuentra el límite Nivel 1

Una biblioteca pide "un sistema para libros". Escribe dos supuestos o restricciones que fijen el alcance.

Pista: Di qué se incluye y qué queda explícitamente fuera.

3.2 Separa las capas Nivel 2

Para una farmacia en línea, escribe un requisito de negocio, uno funcional, uno no funcional y uno de transición.

Pista: Negocio = objetivo; funcional = acción; no funcional = cualidad con un número; transición = temporal.

Lección 4 · 25 min

04 Modelar software: de los diagramas de contexto a UML

Un modelo es una representación simplificada de un sistema que ayuda a razonar sobre él. Los buenos modelos tienen cinco cualidades: son abstractos (dejan fuera lo que no importa), comprensibles, precisos (fieles al sistema), predictivos (permiten conclusiones correctas) y más baratos que construir y estudiar el sistema real. Los modelos se convierten en el lenguaje común entre usuarios, analistas y desarrolladores.

Los modelos de análisis describen el problema en el mundo real: qué hace el negocio. Los modelos de diseño describen la solución de software: cómo lo hará el sistema. La ingeniería directa va de un modelo a código funcional; la ingeniería inversa reconstruye un modelo a partir de un producto o código existente, algo común cuando falta documentación.

Al inicio de un proyecto, un diagrama de contexto dibuja el sistema como una sola caja rodeada de los actores externos y los datos que entran y salen; fija los límites. Después, un diagrama de flujo de datos divide el proceso en pasos con entradas y salidas. Para los datos, un modelo entidad-relación muestra entidades, sus atributos y relaciones con cardinalidad: uno a uno, uno a muchos o muchos a muchos. En una tabla, la clave primaria identifica cada fila y una clave foránea guarda la clave primaria de otra tabla para relacionarlas. Una máquina de estados muestra los estados que puede tener un componente, como Borrador, Pagado y Enviado, y los eventos que lo hacen cambiar. Un diccionario de datos define cada dato, para que todos usen el mismo significado.

UML, el Lenguaje Unificado de Modelado, estandariza diagramas para el desarrollo orientado a objetos. Los diagramas de estructura muestran las partes de un sistema: clases, componentes, objetos, paquetes y despliegue. Los diagramas de comportamiento e interacción muestran cómo actúa: casos de uso, actividades, estados y secuencia. Rara vez necesitas todos. Elige el diagrama que responde la pregunta que tu equipo se hace en ese momento.

> **Piénsalo así**
>
> El mapa de una ciudad es un modelo. Ignora el color de cada casa (abstracto) pero acierta con las calles (preciso), así que puedes planear una ruta antes de caminar (predictivo y barato).



Diagrama · Modelar software: de los diagramas de contexto a UML

Ideas clave

- Un modelo es útil cuando es más simple que el sistema pero sigue siendo correcto en lo que importa.
- Diagrama de contexto = límites; modelo ER = datos; máquina de estados = ciclo de vida de una cosa.
- UML tiene diagramas de estructura y de comportamiento; elige según la pregunta que necesitas responder.

Errores comunes

- ✗ Dibujar todos los diagramas UML "para estar completo".
  ✓ **Solución:** Dibuja el diagrama que responde la pregunta actual.

Ejercicios

4.1 Dibuja un diagrama de contexto con palabras Nivel 1

Para una app de comida a domicilio, enumera los actores externos y un flujo de datos de entrada y salida para cada uno.

Pista: Piensa en clientes, restaurantes, repartidores y la pasarela de pago.

4.2 Diseña una máquina de estados Nivel 3

Enumera los estados de un pedido en línea y el evento de cada transición.

Pista: Empieza en Creado; termina en Entregado o Cancelado.

05 Lección 5 · 15 min

Gestión de la configuración del software

La gestión de la configuración del software (GCS) controla cómo cambian las partes de un sistema con el tiempo. Sus elementos no son solo código fuente: los documentos de requisitos, los diseños, los scripts de prueba, los archivos de compilación y los manuales son elementos de configuración. El objetivo es saber exactamente qué versión de cada elemento pertenece a qué versión del producto.

La GCS se apoya en cuatro ideas. La identificación nombra cada elemento y cada versión. El control de versiones registra cada cambio. El control de cambios decide qué cambios se aprueban y cuándo. La contabilidad de estado informa qué hay en cada línea base. Después, una auditoría comprueba que la línea base coincide con lo que se pretendía.

Sin GCS, los equipos tienen dificultades con preguntas como "¿qué compilación está en producción?" o "¿por qué esta prueba pasó ayer?". Mantener una línea base estable permite que las personas sigan trabajando en paralelo sin sobrescribir el trabajo de otros.

Piénsalo así

Una biblioteca mantiene un catálogo de cada edición y de quién la prestó. La GCS hace lo mismo con un producto: cada elemento, versión y cambio tiene un registro que puedes consultar.

Ideas clave

- Los elementos de configuración incluyen todo lo que forma el producto, no solo el código.
- La GCS responde: qué versión, qué cambio, aprobado por quién y en qué línea base.

Ejercicios

5.1 Inventaría los elementos de configuración Nivel 1

Enumera cinco elementos de configuración de una app móvil que no sean código fuente.

Pista: Piensa en documentos, pruebas, configuración de compilación y versiones.

5.2 Rastrea un error hasta una línea base Nivel 3

Producción muestra un error que no estaba en la compilación de prueba. Describe las cuatro preguntas de la GCS que responderías para hallar la causa.

Pista: Identificación, control de versiones, control de cambios, contabilidad de estado.

06 Lección 6 · 20 min
Control de versiones

Un sistema de control de versiones (VCS) registra los cambios de los archivos a lo largo del tiempo. Ofrece un historial de qué cambió, cuándo y quién lo cambió; la posibilidad de recuperar versiones anteriores; y una forma para que varias personas trabajen en el mismo proyecto sin perder el trabajo de las demás.

Los sistemas centralizados mantienen un repositorio maestro en un servidor. Los sistemas distribuidos, como Git, dan a cada desarrollador una copia local completa del repositorio y de su historial. Puedes hacer commits, crear ramas y consultar el historial sin conexión, y compartir cambios cuando estés conectado. Git es un VCS distribuido. GitHub, GitLab y servicios similares alojan repositorios Git y añaden funciones de colaboración; no son Git en sí.

Un commit es una instantánea del proyecto con un mensaje. Los buenos commits representan un solo cambio lógico, y sus mensajes dicen qué cambió en modo imperativo, por ejemplo "add sum functionality" o "fix off-by-one in pagination". Los commits pequeños y frecuentes hacen que el historial sea más fácil de leer y de revertir.

Piénsalo así

El control de versiones es como un videojuego con puntos de guardado. Puedes volver a cualquier punto, ver qué cambió entre dos y dejar que tus amigos jueguen desde su propia copia.

Ideas clave

- Un VCS guarda el historial, permite recuperar versiones y muestra quién cambió qué y cuándo.
- Git es distribuido; GitHub es un servicio de alojamiento para repositorios Git.
- Un commit debe contener un solo cambio lógico con un mensaje claro.

Errores comunes

- ✗ Escribir mensajes como "cosas" o "cambios".
  ✓ **Solución:** Describe el cambio en una línea imperativa de unos 50 caracteres.

Ejercicios

6.1 Corrige el mensaje de commit Nivel 1

Reescribe estos mensajes en estilo imperativo: "cosas", "Fixed the bug.", "Added a new function for sums".

Pista: Empieza con un verbo, en minúscula, sin punto final, unos 50 caracteres.

6.2 Divide un commit Nivel 2

Un commit mezcla una corrección, un cambio de nombre y una prueba nueva. Describe cómo lo dividirías en tres commits lógicos.

Pista: Prepara los archivos relacionados juntos y haz commit de cada grupo por turno.

07 Lección 7 · 30 min

Fundamentos de Git: áreas, commits y ramas

Git tiene tres áreas. El directorio de trabajo contiene los archivos que editas. El área de preparación (también llamada índice) contiene los cambios que elegiste para el siguiente commit. El repositorio, guardado en la carpeta oculta .git, contiene el historial confirmado. La mayoría de los comandos mueven cambios entre estas áreas: git add lleva el trabajo a la preparación, y git commit lleva lo preparado al repositorio.

El ciclo diario es: editar, revisar, preparar, confirmar. Ejecuta git status con frecuencia; muestra qué archivos están modificados, cuáles están preparados y cuáles no tienen seguimiento. Usa git log --oneline --graph para leer el historial. Usa git diff para ver los cambios no preparados y git diff --staged para ver lo que contendrá el siguiente commit.

Una rama es un puntero móvil a un commit. Crear una rama no copia archivos. Te permite desarrollar una funcionalidad sin cambiar main. Cuando la funcionalidad está lista, la fusionas de vuelta. Si ambas ramas cambiaron las mismas líneas, Git se detiene y marca un conflicto: el archivo contiene marcadores <<<<<<<, ======= y >>>>>>> que muestran las dos versiones. Editas el archivo hasta el resultado deseado, lo preparas y haces commit.

Piénsalo así

Git es un estudio fotográfico. El directorio de trabajo es el set, la preparación es elegir qué fotos van al álbum, y el commit es imprimir la página del álbum. Una rama es un segundo álbum que puedes armar sin tocar el primero.



Diagrama · Fundamentos de Git: áreas, commits y ramas

Ideas clave

- Edita en el directorio de trabajo, prepara con git add y registra con git commit.
- git status y git log muestran dónde estás; úsalos antes y después de cada paso.
- Una rama es un puntero. Fusionar combina historiales; los conflictos requieren una decisión humana.

Errores comunes

- ✗ Hacer commit sin revisar git status, incluyendo archivos no relacionados.
  ✓ **Solución:** Ejecuta git status y git diff --staged antes de cada commit.
- ✗ Borrar los marcadores de conflicto sin elegir el contenido correcto.
  ✓ **Solución:** Decide el contenido final, quita las tres líneas de marcador, luego prepara y haz commit.

Ejercicios

7.1 Predice el estado Nivel 1

Editas a.txt, ejecutas git add a.txt y luego editas a.txt otra vez. ¿La segunda edición está preparada? ¿Qué muestra git status?

Pista: La preparación captura el archivo en el momento de ejecutar git add.

7.2 Lee el grafo Nivel 2

Ejecuta git log --oneline --graph --all en la carpeta de prueba tras el Laboratorio 2. Explica en una frase por qué main y feature-sum apuntan al mismo commit.

Pista: ¿Qué pasa cuando la rama destino no tiene commits nuevos?

7.3 Resuelve según la intención Nivel 3

Dos ramas cambiaron la misma función: una renombró un parámetro y la otra añadió validación. Describe la resolución que quieres y los comandos.

Pista: Conserva ambas intenciones, luego prepara y haz commit.

08 Lección 8 · 25 min
GitHub, remotos y flujos de colaboración

Un remoto es un enlace con nombre a otro repositorio, que por convención se llama origin. git remote add origin <url> lo registra. git push -u origin main sube tu rama main y configura el upstream. git fetch descarga commits nuevos sin cambiar tus archivos; git pull hace fetch y luego integra esos commits en tu rama actual.

Un flujo de trabajo habitual en equipo usa ramas de funcionalidad con solicitudes de cambio (llamadas merge requests en GitLab). Creas una rama, la subes y abres una solicitud. Quienes revisan leen el diff, comentan y aprueban. Solo después de la aprobación la rama se fusiona con main. El paso de revisión es donde los equipos detectan defectos y comparten conocimiento.

La buena colaboración también significa mantener main estable, actualizar tu copia local antes de empezar a trabajar y escribir commits que quien revisa pueda entender uno por uno.

Piénsalo así

Una solicitud de cambio es como un borrador enviado a un compañero antes de entregar el trabajo. El revisor no lo escribe por ti, pero detecta errores mientras son fáciles de corregir.



Cada círculo es un commit. Una rama es un puntero. Fusionar une historiales.

Diagrama · GitHub, remotos y flujos de colaboración

Ideas clave

- origin es el nombre habitual del remoto que clonaste o conectaste.
- fetch descarga; pull descarga e integra; push sube.
- Las solicitudes de cambio hacen de la revisión un paso planificado antes de llegar a main.

Errores comunes

- ✗ Subir cambios directamente a main en un proyecto compartido.
  ✓ **Solución:** Trabaja en una rama de funcionalidad y abre una solicitud de cambio para revisión.

Ejercicios

8.1 Ordena el flujo de trabajo Nivel 1

Ordena estos pasos: fusionar, abrir solicitud de cambio, subir la rama, revisar y aprobar, crear la rama.

Pista: No puedes abrir una solicitud antes de que la rama exista en el remoto.

8.2 Recupérate de un push rechazado Nivel 3

Un compañero subió cambios a main mientras trabajabas. Tu git push fue rechazado. Escribe la secuencia segura de comandos.

Pista: Primero fetch, luego integra y luego push. Nunca fuerces.

Soluciones resueltas

1.1 Encuentra la brecha. Ejemplo: "¿Qué reporte, y cuánto tarda hoy frente a la meta?" y "¿Quién lo usa y con qué frecuencia?". Cada respuesta convierte un deseo vago en un requisito comprobable.

1.2 Escribe un requisito comprobable. Ejemplo: "Tras 5 intentos fallidos en 10 minutos, la cuenta se bloquea 15 minutos y el usuario recibe un correo." Ahora un tester puede comprobarlo.

2.1 Elige un modelo. Puede encajar un proceso secuencial (tipo cascada), porque los requisitos son estables y los errores tempranos serían caros. Un proceso iterativo también sirve si el equipo quiere retroalimentación temprana de la interfaz.

2.2 Dibuja tu propio historial de líneas base. La línea base 2 es inicio de sesión más mensajes, funcional y documentada. El incremento es la función de mensajes comparada con la línea base 1.

3.1 Encuentra el límite. Ejemplo: "Incluye préstamos y devoluciones para socios." "Excluye la compra de libros nuevos y el pago de salarios." Los límites claros evitan problemas de alcance.

3.2 Separa las capas. Negocio: "aumentar las ventas en línea un 20% este año." Funcional: "los clientes suben una foto de la receta." No funcional: "los pedidos se confirman en menos de 5 segundos." Transición: "importar 10 años de registros de clientes del sistema anterior."

4.1 Dibuja un diagrama de contexto con palabras. Cliente: pedido entra, estado sale. Restaurante: menú entra, pedido sale. Repartidor: ubicación entra, tarea de recogida sale. Pasarela de pago: confirmación entra, solicitud de cobro sale.

4.2 Diseña una máquina de estados. Creado --pagar--> Pagado --empacar--> Empacado --enviar--> Enviado --entregar--> Entregado. Creado o Pagado --cancelar--> Cancelado. Cada flecha es un evento; cada caja es un estado.

5.1 Inventaría los elementos de configuración. Ejemplos: especificación de requisitos, archivos de diseño de la interfaz, plan y scripts de prueba, archivos de configuración de compilación, notas de versión, manual de usuario.

5.2 Rastrea un error hasta una línea base. ¿Qué versiones exactas están en producción (identificación)? ¿Qué cambió entre la compilación de prueba y producción (control de versiones)? ¿Qué cambios se aprobaron y quién (control de cambios)? ¿La compilación de producción es la registrada en la línea base (contabilidad de estado)?

6.1 Corrige el mensaje de commit. fix login redirect loop | fix bug … (nombra el error) | add sum function. Los verbos específicos ayudan a leer el historial rápido.

6.2 Divide un commit. Prepara los archivos de la corrección y haz commit; prepara los archivos renombrados y haz commit; prepara la prueba y haz commit. Ahora cada commit se puede revisar o revertir por separado.

7.1 Predice el estado. No: la segunda edición no está preparada. git status muestra a.txt en "Changes to be committed" (la primera versión) y en "Changes not staged for commit" (la edición nueva).

7.2 Lee el grafo. main no tenía commits nuevos desde que se separó feature-sum, así que Git movió el puntero hacia adelante (fast-forward) en lugar de crear un commit de fusión.

7.3 Resuelve según la intención. Edita la función para usar el nuevo nombre del parámetro e incluir la validación. Quita todos los marcadores, ejecuta git add sobre el archivo y luego git

commit para terminar la fusión.

8.1 Ordena el flujo de trabajo. Crear la rama → subir la rama → abrir la solicitud de cambio → revisar y aprobar → fusionar.

8.2 Recupérate de un push rechazado. git fetch origin, luego git pull origin main (o git rebase origin/main), resuelve conflictos, ejecuta pruebas y luego git push origin main.

Glosario

Pregunta	Respuesta
¿Cuál es la diferencia principal entre programar e ingeniería de software?	Programar escribe código. La ingeniería de software también planifica, especifica, valida y mantiene un sistema que cubre una necesidad real.
¿Por qué los ingenieros construyen modelos antes de escribir código?	Los modelos son más baratos de cambiar que el código. Simplifican el sistema, facilitan la discusión y permiten predecir su comportamiento.
¿Qué define un proceso de desarrollo de software?	Qué se hace, quién, cuándo y cómo. Sus cuatro actividades comunes son especificación, desarrollo, validación y evolución.
Cascada frente a iterativo e incremental: ¿cuál es la diferencia clave?	Cascada ejecuta cada fase una vez, en orden. El trabajo iterativo repite las fases en ciclos cortos y entrega incrementos funcionales.
¿Qué es una línea base?	Una versión acordada y almacenada del sistema y sus documentos. El trabajo posterior se compara con ella.
¿Qué es un incremento?	La diferencia entre dos líneas base sucesivas.
Nombra tres tipos de elemento de configuración además del código fuente.	Cualquiera de tres: documentos de requisitos, diseños, scripts de prueba, archivos de compilación, manuales.
Control de cambios frente a contabilidad de estado	El control de cambios decide qué cambios se aprueban. La contabilidad de estado informa qué hay en cada línea base.
Da dos razones para usar un sistema de control de versiones.	Conserva el historial, recupera versiones anteriores, muestra quién cambió qué y cuándo, y permite trabajar en paralelo.
Control de versiones centralizado frente a distribuido	Centralizado: un repositorio maestro en un servidor. Distribuido: cada persona tiene una copia local completa con historial, como en Git.
Git frente a GitHub	Git es la herramienta de control de versiones. GitHub es un servicio de alojamiento para repositorios Git con revisión y colaboración.
¿Qué es un buen commit?	Un cambio lógico, pequeño y frecuente, con un mensaje imperativo como "add sum functionality".
Nombra las tres áreas de Git.	Directorio de trabajo, área de preparación (index) y repositorio (.git).
¿Qué hace git add?	Mueve cambios del directorio de trabajo al área de preparación. No crea un commit.
¿Qué hace git commit -m "mensaje"?	Registra los cambios preparados como un nuevo commit con ese mensaje.
¿Por qué ejecutar git status antes de hacer commit?	Muestra archivos modificados, preparados y sin seguimiento, para no registrar cambios no relacionados.
¿Qué muestra git log --oneline --graph?	Un historial compacto de commits con hashes cortos, mensajes y líneas de ramas.

¿Qué es una rama en Git?	Un puntero móvil a un commit que permite desarrollar trabajo aislado. Crear una rama no copia archivos.
¿Qué causa un conflicto de fusión?	Dos ramas cambiaron las mismas líneas de forma distinta. Git no puede decidir, así que una persona debe elegir el resultado.
¿Qué es origin?	El nombre habitual de un repositorio remoto, como la copia de GitHub desde la que clonaste.
git fetch frente a git pull	fetch descarga commits remotos nuevos sin cambiar tus archivos. pull los descarga e integra en tu rama actual.
¿Qué hace git push -u origin main?	Sube tu rama main local a origin y configura el upstream, así los siguientes push y pull no necesitan argumentos.
¿Por qué usar solicitudes de cambio (pull requests)?	Hacen de la revisión un paso planificado: revisores leen el diff, comentan y aprueban antes de fusionar la rama en main.
¿Por qué es arriesgado subir directamente a una main compartida?	Los cambios sin probar ni revisar llegan a todos a la vez, y el historial de ramas compartidas no debe reescribirse.
¿Qué hace que un enunciado sea un buen requisito?	Es una capacidad o condición que el sistema debe cumplir, y alguien puede comprobar si se cumple.
Nombra las siete actividades de la ingeniería de requisitos.	Inicio, obtención, elaboración, negociación, especificación, validación y gestión.
¿Tres motivos por los que la obtención es difícil?	Alcance (límites poco claros), comprensión (los usuarios no saben expresar lo que necesitan), volatilidad (las necesidades cambian).
Requisito funcional frente a no funcional	Funcional: un comportamiento ("enviar un recibo"). No funcional: una cualidad con medida ("menos de 2 segundos").
¿Qué es un requisito de transición?	Una capacidad temporal para pasar del estado anterior al nuevo, como migrar datos antiguos.
¿Cinco cualidades de un buen modelo?	Abstracto, comprensible, preciso, predictivo y barato.
Modelo de análisis frente a modelo de diseño	El análisis describe el problema del mundo real. El diseño describe la solución de software.
¿Qué muestra un diagrama de contexto?	El sistema como una caja, los actores externos alrededor y los datos que entran y salen. Fija los límites.
Clave primaria frente a clave foránea	La clave primaria identifica cada fila de una tabla. La clave foránea guarda la clave primaria de otra tabla para relacionarlas.
Diagramas UML de estructura frente a comportamiento	Estructura: clases, componentes, paquetes, despliegue. Comportamiento: casos de uso, actividades, estados, secuencia.

Cuestionario de práctica

1. ¿Qué elemento es un elemento de configuración en la gestión de la configuración del software?

A. Un documento de requisitos
B. La cafetera de la oficina
C. Un calendario personal

2. ¿Qué afirmación sobre Git es correcta?

A. Git es un sistema de control de versiones distribuido; cada clon contiene el historial completo.
B. Git no puede registrar un commit sin una conexión a internet a GitHub.
C. Git y GitHub son dos nombres para la misma herramienta.

3. En un proceso iterativo e incremental, ¿qué representa la diferencia entre dos líneas base sucesivas?

A. Un incremento
B. Un registro de riesgos
C. Solo la entrega final

4. Verdadero o falso: Git y GitHub son el mismo producto.

A. Verdadero
B. Falso

5. Verdadero o falso: git add crea un commit en el historial del repositorio.

A. Verdadero
B. Falso

6. Verdadero o falso: crear una rama de Git copia todos los archivos del proyecto.

A. Verdadero
B. Falso

7. Los cambios elegidos para el siguiente commit esperan en el área de ____. (Pista: también se llama índice.)

Respuesta: _____

8. Escribe un comando que cree y cambie a una rama llamada feature-login (sintaxis moderna).

Respuesta: _____

9. En dos frases, explica la diferencia entre git fetch y git pull.

Respuesta: _____

10. Menciona un beneficio de los commits pequeños y enfocados.

Respuesta: _____

11. Relaciona cada comando con su propósito.

Relaciona cada elemento de la izquierda con uno de la derecha.

1. git add	A. Muestra archivos modificados, preparados y sin seguimiento
2. git commit	B. Mueve los cambios al área de preparación
3. git status	C. Descarga commits remotos sin fusionarlos
4. git fetch	D. Registra los cambios preparados como un commit nuevo

12. Relaciona cada modelo de proceso con su característica principal.

Relaciona cada elemento de la izquierda con uno de la derecha.

1. Cascada	A. Ciclos cortos con retroalimentación frecuente de los usuarios
2. Iterativo e incremental	B. Las fases se ejecutan una vez, en orden
3. Adaptativo (por ejemplo, Scrum)	C. Iteraciones repetidas que producen líneas base

---

13. Interpreta los comandos. ¿Qué informa git status sobre notes.txt?

```
echo "v2" >> notes.txt
git add notes.txt
git status
```

A. notes.txt aparece en "Changes to be committed"
B. notes.txt ya está en el historial del repositorio
C. notes.txt no ha cambiado

14. Interpreta el comando. ¿Qué mostrará la salida?

```
git log --oneline -3
```

A. Los tres commits más recientes, uno por línea, con hashes cortos y mensajes
B. Los tres archivos más antiguos del proyecto
C. Las tres ramas con más conflictos

15. Un compañero dice que la última versión se comporta incorrectamente. ¿Qué pregunta harías primero?

A. ¿Qué elementos de configuración y versiones están en esa línea base?
B. ¿Quién tiene la computadora más rápida?
C. ¿Podemos borrar el repositorio y empezar de nuevo?

16. Quieres que un compañero revise tu cambio de inicio de sesión antes de que llegue a main. ¿Qué haces?

A. Subo una rama de funcionalidad y abro una solicitud de cambio para revisión
B. Subo directamente a main y pido comentarios después
C. Envío por correo un zip del proyecto

17. Dos ramas cambiaron la misma línea de app.js. La fusión se detiene. ¿Qué pasó?

A. Un conflicto de fusión; una persona debe elegir el contenido final
B. Git borró el repositorio
C. El remoto está desconectado

18. git push es rechazado porque el remoto tiene commits que no tienes localmente. ¿Cuál es la solución más segura?

A. Ejecutar git fetch, integrar los cambios remotos (merge o rebase) y luego volver a hacer push
B. Ejecutar git push --force para sobrescribir el remoto
C. Borrar tu repositorio local y clonar de nuevo

19. git commit falla con "Author identity unknown". ¿Qué debes hacer?

A. Configurar tu nombre y correo con git config --global user.name y user.email
B. Borrar la carpeta .git
C. Hacer commit con git add -f

20. Después de una fusión, un archivo todavía contiene líneas <<<<<<< HEAD. ¿Cuál es la resolución correcta?

A. Editar el archivo con el contenido deseado, quitar las líneas de marcador, hacer git add del archivo y luego commit
B. Dejar los marcadores y hacer commit de todos modos
C. Ejecutar git merge --abort después de hacer commit

21. ¿Qué enunciado es un requisito no funcional?

A. Los resultados de búsqueda aparecen en menos de un segundo
B. Los usuarios pueden restablecer su contraseña por correo
C. Los administradores pueden exportar un informe de ventas

22. Verdadero o falso: los requisitos de transición permanecen en el sistema para siempre.

A. Verdadero
B. Falso

23. En cada reunión, un usuario distinto cambia lo acordado la semana anterior. ¿Qué problema de obtención es y qué ayuda?

A. Volatilidad: registrar acuerdos en actas y trabajar con un usuario representativo
B. Alcance: comprar un servidor más rápido
C. Comprensión: empezar a programar de inmediato

24. Relaciona cada actividad con lo que hace.

Relaciona cada elemento de la izquierda con uno de la derecha.

1. Obtención	A. Revisa la especificación con los usuarios antes de construir
2. Negociación	B. Recoge las necesidades de los interesados
3. Validación	C. Registra cada cambio de los requisitos
4. Gestión	D. Concilia visiones en conflicto, costo y riesgo

---

25. ¿Qué modelo muestra mejor los límites de un sistema y quién intercambia datos con él?

A. Diagrama de contexto
B. Máquina de estados
C. Diagrama de clases

26. Una columna que guarda la clave primaria de otra tabla para relacionarlas se llama clave ____.

Respuesta: _____

Clave de respuestas

1. A — Cualquier artefacto que forme parte del producto o de su documentación puede ser un elemento de configuración. Los requisitos son un ejemplo central.
2. A — Cada clon tiene el repositorio y el historial completos, así que puedes hacer commits sin conexión.
3. A — Por definición, la diferencia entre dos líneas base es un incremento.
4. B — Falso. Git es la herramienta de control de versiones. GitHub aloja repositorios Git y añade funciones de colaboración.
5. B — Falso. git add solo prepara los cambios. git commit los registra.
6. B — Falso. Una rama es un puntero a un commit. Git no copia los archivos al crear una.
7. El área de preparación, también llamada índice, contiene los cambios que irán en el siguiente commit.
8. git switch -c crea y cambia de rama. git checkout -b hace lo mismo con la sintaxis anterior.
9. fetch descarga los commits remotos sin cambiar tu rama de trabajo. pull hace fetch y luego los integra en tu rama actual.
10. Los commits pequeños son más fáciles de revisar, entender y revertir si un solo cambio rompe algo.
11. 1→B, 2→D, 3→A, 4→C — add prepara, commit registra, status informa y fetch descarga commits remotos sin cambiar tus archivos.
12. 1→B, 2→C, 3→A — Cascada es secuencial; el trabajo iterativo produce líneas base; los métodos adaptativos hacen ciclos cortos de retroalimentación.
13. A — El cambio se prepara con git add, así que git status lo lista como listo para commit. Nada entra al historial hasta que se ejecuta git commit.
14. A — --oneline muestra un commit por línea, y -3 limita la salida a tres commits, del más reciente al más antiguo.
15. A — Antes de depurar, identifica exactamente qué versiones se publicaron. La GCS hace que esto sea trazable.
16. A — Una rama de funcionalidad con solicitud de cambio permite que quien revisa vea el diff y apruebe antes de fusionar.
17. A — Git no puede elegir entre dos ediciones distintas de las mismas líneas, así que marca el conflicto para que lo resuelvas.
18. A — Integra primero el trabajo remoto. Un push forzado puede borrar los commits de un compañero y debe evitarse en ramas compartidas.
19. A — Git guarda el nombre y el correo del autor en cada commit, así que ambos deben configurarse primero.
20. A — Los marcadores no son código válido. Elige el contenido final, quita todos los marcadores, prepara el archivo y completa la fusión con un commit.
21. A — Describe una cualidad (velocidad) con una medida. Los otros describen comportamientos, así que son funcionales.
22. B — Falso. Son temporales; solo hacen falta para pasar de la situación anterior a la nueva.
23. A — Los requisitos que cambian constantemente son volátiles. Los acuerdos escritos y un representante único los estabilizan.
24. 1→B, 2→D, 3→A, 4→C — La obtención recoge, la negociación concilia, la validación revisa y la gestión registra los cambios.
25. A — Un diagrama de contexto dibuja el sistema como una caja con sus actores externos y flujos de datos.

26. Es una clave foránea. Crea relaciones como un cliente con muchos pedidos.